

Implementation Plan & Workflow

Student Expense Tracker with Analytics | Team 17 | SWE 4637

User Stories (Why We're Building This)

Simple explanations of what the app does, from the point of view of a real student using it — no technical terms needed.

User Story 1: Tracking Daily Spending

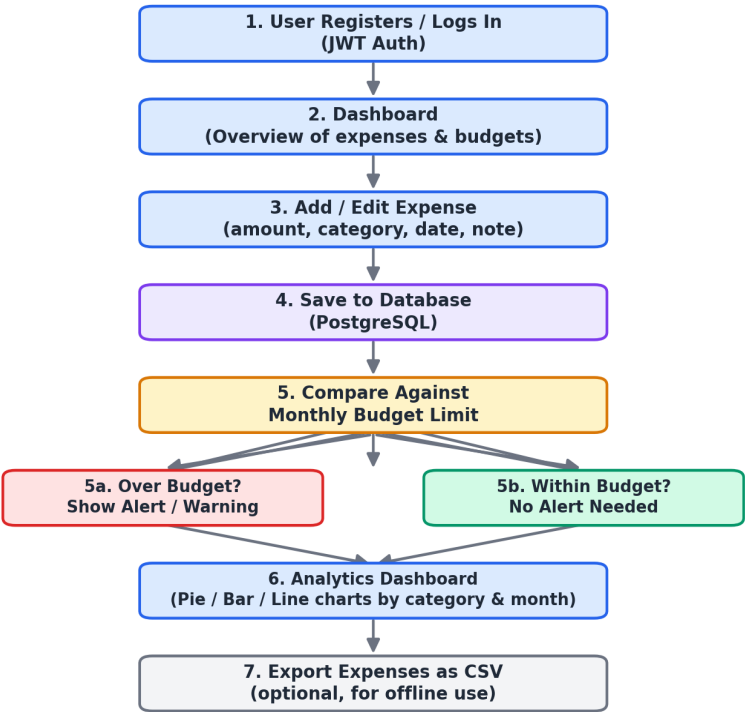
"As a university student, I want to log every expense I make (like food, transport, or books) under a category, so that at the end of the month I can clearly see where my money went instead of wondering why I'm broke."

User Story 2: Staying Within Budget

"As a student living on a fixed monthly allowance, I want to set a spending limit for categories like 'Food' or 'Entertainment' and get warned when I'm about to cross it, so that I don't run out of money before the month ends."

App Workflow (Block Diagram)

This shows the journey of a single expense, from the moment a user logs in to the moment they see it reflected in charts.



Implementation Steps & Timeline

A single ordered checklist — work through it top to bottom. Times assume roughly a 12-hour build day starting at 9:00 AM; shift the clock times if you start later.

9:00 – 9:30 AM	Step 1: Project Setup
• Create the GitHub repo and folder structure (backend/ and frontend/) • Install required tools: Python, Node.js, PostgreSQL • Create the database and confirm you can connect to it	
9:30 – 10:30 AM	Step 2: Database Design
• Create the 4 tables: users, categories, expenses, budgets • Set up SQLAlchemy models matching the tables • Run the first migration with Alembic to confirm tables are created	
10:30 – 11:30 AM	Step 3: User Authentication
• Build register and login API endpoints • Hash passwords securely and generate JWT tokens on login • Test login/register using Postman or Thunder Client	
11:30 AM – 1:00 PM	Step 4: Expense Management (Backend)
• Build Create, Read, Update, Delete (CRUD) APIs for expenses • Build CRUD APIs for categories • Protect these routes so only logged-in users can access them	
1:00 – 1:45 PM	Lunch Break
1:45 – 3:00 PM	Step 5: Budget & Alert Logic
• Build API to set a monthly budget per category • Add logic to compare total spending vs. the budget limit • Return a warning flag in the API response when a budget is close to or over the limit	
3:00 – 4:30 PM	Step 6: Frontend Foundation
• Set up the React app with Tailwind CSS • Build Login and Register pages, connect them to the backend • Build the basic Dashboard layout and navigation	
4:30 – 6:00 PM	Step 7: Expense & Budget Screens
• Build the 'Add Expense' form and the expense list view • Build the 'Set Budget' form and show the overspend warning in the UI • Connect all forms and lists to the backend APIs	
6:00 – 7:30 PM	Step 8: Analytics Dashboard

- Build an API endpoint that returns totals grouped by category and by month
- Add a pie chart (spending by category) using Recharts — this is the priority chart
- If time allows, add a bar chart and a line chart for trends over time

7:30 – 8:00 PM

Step 9: CSV Export (Optional)

- Build a backend endpoint that returns expenses as a downloadable CSV file
- Add an 'Export' button on the frontend

8:00 – 8:45 PM

Step 10: Testing & Bug Fixing

- Walk through the entire app as a real user: register → log in → add expense → set budget → view charts
- Fix any errors, broken links, or crashes found along the way

8:45 – 9:00 PM

Step 11: Final Check & Demo Prep

- Add a few sample expenses so the app looks populated for the demo
- Do a final run-through before presenting